GENESIS 2.5: opt-in OpenCL and CUDA acceleration for the GENESIS/PGENESIS compartmental neural simulator

Karol Chlasta[a,*], Grzegorz Marcin Wójcik[b]

[a]*Kozminski University, ul. Jagiellońska 57/59, 03-301 Warsaw, Poland; WarsawIQ, Al. Jana Pawła II 43A/37B, 01-001 Warsaw, Poland*
[b]*Maria Curie-Skłodowska University, ul. Akademicka 9, 20-033 Lublin, Poland*

Abstract

GENESIS is a long-established compartmental neural simulator; PGENESIS extends it to MPI execution. We release GENESIS 2.5, an accelerator-enabled extension of GENESIS 2.4/PGENESIS 2.4: it adds opt-in OpenCL and CUDA backends for the Hodgkin–Huxley channel-update path while leaving every existing CPU, MPI, model, and SLI-script workflow unchanged. A multi-step ("multiloop") GPU kernel batches K simulation steps into a single dispatch. Timed by wall clock on identically instrumented CPU and GPU arms (integrated AMD Radeon 890M, fp32 kernel vs. fp64 CPU, $K = 50\,000$ steps of driven spiking neurons), and after a scheduler fix that removes an $O(N)$ per-step overhead, the accelerated *simulation phase* peaks near $N = 5000$ neurons at $\sim$18$\times$ (GPU in its sustained clock state; $\sim$13$\times$ for a single cold-start run), reaching $\sim 1.5 \times 10^9$ neuron-steps/s; the full end-to-end run, including one-time CPU-bound model construction, is up to $\sim$4$\times$ faster. The fp32 accelerator matches the fp64 CPU to $\sim 2 \times 10^{-7}$ V over a single action potential. The CUDA backend, a faithful fp32 port on the same interface, is validated on an NVIDIA RTX 4090: it reproduces the fp64 CPU action potential to 7×10^{-8} V and reaches 5–132$\times$ step-phase speedup for $N = 500$–$50\,000$ neurons, the speedup rising with network size as the desktop GPU is progressively saturated. Alongside the accelerator we fix three independent, previously unnoticed defects in the GENESIS Hines solver that silently suppressed `inject`-driven voltage transients in any multi-neuron `hsolve` configuration, GPU or CPU-only.

*Corresponding author.
Email addresses: `karol@chlasta.pl` (Karol Chlasta), `gmwojcik@umcs.edu.pl` (Grzegorz Marcin Wójcik)

Keywords: GENESIS, PGENESIS, OpenCL, GPU acceleration, compartmental neural simulation, Hines solver

---

Required Metadata

Current code version

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v2.5 (2026-07-04; OpenCL + CUDA-validated)
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/KarolChlasta/genesis-2.4`
C3	Legal Code License	GNU General Public License v2 or later (program); GNU Lesser General Public License v2.1 or later (library portions) – see `LICENSE`
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	C, OpenCL C, CUDA C++ (accelerator backends), MPI (PGENESIS), Python (benchmark analysis/plotting)
C6	Compilation requirements, operating environments & dependencies	Linux x86_64, GCC, GNU make, an OpenCL 1.2+ runtime (tested: ROCm 6.3.1 and Mesa rusticl) or a CUDA 12.x toolkit (tested: CUDA 12.8, `sm_89`) for the GPU backends, an MPI implementation for PGENESIS
C7	If available Link to developer documentation/manual	`README.md` and `paper/REPLICATION.md` in the repository
C8	Support email for questions	karol@chlasta.pl

Table 1: Code metadata (mandatory)

Main text

1. Motivation and significance

Compartmental neural simulators remain essential for biophysically detailed models where channel kinetics and dendritic geometry matter and cannot be abstracted away. GENESIS [1] is one of the longest-maintained simulators in this class, and PGENESIS extends it to MPI-distributed execution for large networks. Both are still used in production modeling workflows, scripted via GENESIS's SLI language, that predate and are independent of newer simulator ecosystems.

Modern workstations increasingly ship capable integrated GPUs, but GENESIS has no accelerator backend: every channel-kinetics update runs on the CPU regardless of available GPU hardware. This is a growing gap, since channel-update kernels (per-compartment gating-variable integration) are an embarrassingly parallel workload well suited to GPU execution, and existing GENESIS/PGENESIS users have no path to accelerator support without abandoning their existing models and SLI scripts.

GENESIS 2.5 closes this gap directly: it is GENESIS 2.4/PGENESIS 2.4 plus an OpenCL backend for the `hsolve` channel-update path, and a CUDA backend on the same interface (a faithful fp32 port of the OpenCL kernels, selected at build time by `-DUSE_CUDA`, and validated on an NVIDIA RTX 4090; see Illustrative examples). No existing CPU or MPI workflow, model file, or SLI script needs to change to benefit from GPU acceleration where it is available, and the same workflows run unmodified where it is not. While building and validating this backend, we additionally found and fixed three independent solver defects in `hines_solve.c`/`hines_init.c` that affect any multi-neuron `hsolve` configuration – including pure-CPU GENESIS deployments. Their effect on the benchmarks reported here is concrete: current injection (`inject`) silently failed to produce a voltage response in this configuration prior to the fix, which would have invalidated any benchmark that compares dynamics rather than only opcode-level throughput, and may have affected unrelated multi-neuron `hsolve` models elsewhere.

We name this release "2.5" rather than "3.0" deliberately. "GENESIS 3" was a separate modularization effort [2] that produced independent successor lines (Neurospaces/Heccer [3], MOOSE) rather than a single installable artifact comparable to GENESIS 2.4, and evolved into the CBI software federation [4, 5, 6]. GENESIS 2.5 is not a re-architecture and does not depend on, or claim continuity with, that line; it targets existing GENESIS 2.4 deployments that should not need to migrate models or scripts to gain accelerator support.

2. Software description

   2.1 Software architecture

GENESIS 2.5 keeps the existing GENESIS/PGENESIS architecture (SLI interpreter, `hsolve` fast compartmental solver, PGENESIS MPI partitioning) unmodified and adds one new component: an OpenCL backend invoked from `hsolve` when an attached `hsolve` element uses `chanmode=4` (or 5) with real ion-channel state. Two dispatch modes are implemented in `ocl_hsolve.c` against the kernel file `genesis/src/hines/opencl/ocl_channel.cl`:

- *Per-step dispatch* (`ocl_chip_update`): one `clEnqueueNDRangeKernel` call per simulation step, with channel state uploaded and downloaded every step.
- *Multiloop dispatch* (`chip_channel_multiloop`, enabled via the environment variable `GENESIS_OCL_MULTILOOP=<K>`): a single kernel dispatch runs all K steps in an inner GPU-side loop, amortizing the per-step PCIe transfer cost that otherwise dominates wall time (see Illustrative examples below).

Device attribution and dispatch are confirmed at run time, not assumed: the command queue is built with `CL_QUEUE_PROFILING_ENABLE`, and every accelerator-attributed benchmark run in this release emits a non-empty `OCL PROFILING SUMMARY` at exit. This guards against two failure modes found during development: an `hsolve` with no attached channels never reaches the kernel at all, and a kernel that fails to build (e.g. for lack of `cl_khr_fp64` on a given device) silently falls back to the CPU path with no GPU work performed. The channel kernel itself is built in `float` (fp32) rather than `double`, so it runs on OpenCL devices/runtimes that expose no fp64 support at all (observed for Mesa rusticl on the AMD Radeon 890M used here); the rest of `hsolve` remains `double` as before, with conversion only at the host/device buffer boundary.

2.2 Software functionalities

- Unmodified serial (`genesis`), headless (`nxgenesis`), and MPI (PGENESIS) execution paths.
- OpenCL channel-update kernel for Hodgkin–Huxley-style `tabchannel` gating (Na X^3Y^1, K X^4), dispatched per-step or batched over K steps via multiloop.
- A CUDA backend on the same call site (`genesis/src/hines/cuda/`): the device kernels are a line-for-line fp32 port of the OpenCL kernels, and a thin C glue mirrors the OpenCL dispatch and multiloop control flow. The GENESIS-facing entry point is identical (`cuda_chip_update` in place of `ocl_chip_update`),

so the same models, SLI scripts, and benchmark harness drive either backend; the accelerator env var and dispatch/profiling reports are shared. The CUDA backend is validated on an NVIDIA RTX 4090 (see Illustrative examples).

- Device-side dispatch confirmation (`OCL/CUDA PROFILING SUMMARY`) distinguishing a genuine GPU run from CPU fallback, so accelerator results in any downstream benchmark are auditable rather than assumed.
- A driven-spiking benchmark (`hh_spiking_benchmark.g`): N Hodgkin–Huxley neurons under one `hsolve`, each injected with a suprathreshold current so they fire action-potential trains – the biophysically relevant regime, and the one that exercises the repaired `inject` path – with a wall-clock harness that times both arms identically and separates one-time model construction from the per-step simulation phase.
- Three corrected defects in the Hines solver (forward-elimination fast-path guard, backward-elimination root handling, `SET_DIAG`/`SKIP_DIAG` row selection) that make multi-neuron `hsolve` configurations produce correct, independent voltage dynamics per neuron under current injection – a precondition for any GPU/CPU dynamical comparison, and a correctness fix independent of GPU support.

3. Illustrative examples

Reproducing the CPU baseline. `nxgenesis -nosimrc -notty -batch` on the bundled `Cal8.g` and `Cal7difshell.g` models gives stable, low-variance timings (CV $\approx 1\%$ over 30 replicates; Fig. 1), establishing that ordinary GENESIS workflows are unaffected by this release.

Reproducing the GPU multiloop result. `genesis/Scripts/benchmark/hh_spiking_benchmark.g` builds N Hodgkin–Huxley neurons under one `hsolve` (`chanmode=4`), each driven by a suprathreshold current so they fire action-potential trains. Running the `nxgenesis` binary with `GENESIS_OCL_MULTILOOP=`K dispatches all K steps in one GPU call. We time both arms by wall clock with identical instrumentation (`experiments/run_overnight_campaign.py`): each configuration is run at $K = 50\,000$ steps and at $K = 0$, and the per-step *simulation phase* is their difference, so one-time model construction (CPU-bound and identical on both arms) is separated from the accelerated stepping. The CPU arm is `nxgenesis_nocl` (built without OpenCL, fp64); the GPU arm is `nxgenesis` (fp32 multiloop, Mesa rusticl, AMD Radeon

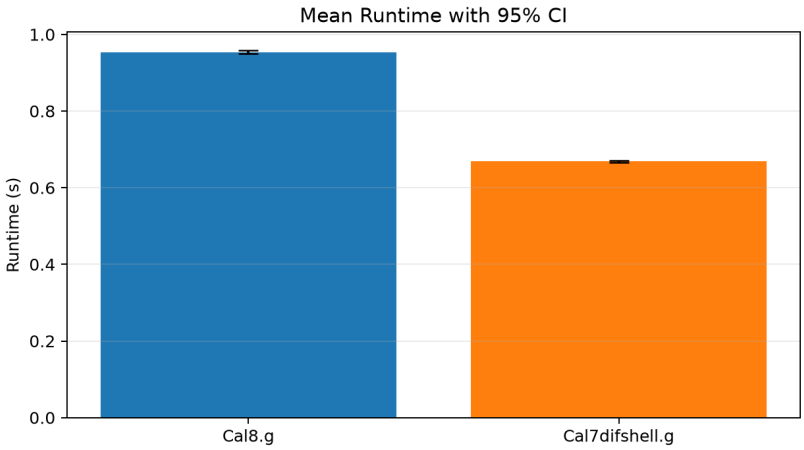

Figure 1: CPU baseline mean wall-clock runtime with 95% CI, `nxgenesis`, 30 replicates per workload.

890M). Both are headless, avoiding the earlier GUI-linked-binary confound. Reps are 30+ at small N; 95% CIs are in `experiments/data/campaign_wallclock_summary.csv`.

Table 2 and Fig. 2 report the result after the scheduler and solver fixes described above. The *step-phase* speedup (the accelerated channel-update simulation, which is what scales with simulated biological duration) peaks near $N = 5000$, at $\sim$18$\times$ with the GPU in its sustained (warm) clock state and $\sim$13.5$\times$ for a single cold-start run; peak GPU throughput is $\sim 1.5 \times 10^9$ neuron-steps/s. The gap between the two is a property of the integrated GPU, not measurement noise: the CPU-bound model build leaves the GPU idle and downclocked, so the first dispatch of a single run is $\sim$1.3–2$\times$ slower than the sustained state that long or repeated runs reach. The sustained dispatch is highly reproducible ($\pm$2% for $N \leq 10\,000$; `experiments/data/campaign_warm_dispatch.csv`), whereas single-run timings vary run-to-run by up to $\sim$2$\times$ (`experiments/data/campaign_reproducibility.csv`). Beyond $N \approx 10\,000$ the integrated GPU no longer sustains its fast clock state under this workload and the speedup falls toward $\sim$3$\times$ regardless of warm-up. The *end-to-end* speedup (including one-time, CPU-bound model construction) peaks at $\sim$4$\times$ for a single run; for longer simulations the construction cost amortizes and end-to-end approaches the step-phase figure (Fig. 4).

Numerical fidelity of the fp32 accelerator. `genesis/Scripts/benchmark/hh1952_ap_verify.g` runs the classic 1952 squid model

N	**CPU step (s)**	**GPU step (s)**	**Step-phase sustained**	**Step-phase single-run**	**Throughput (M nsteps/s)**
500	0.302	0.112	2.7×	2.0×	223
1 000	0.587	0.112	5.2×	3.7×	446
2 000	1.166	0.117	10.0×	6.5×	855
5 000	2.965	0.164	**18.1**×	13.5×	**1524**
10 000	5.799	0.406	14.3×	6.8×	1232
20 000	11.918	3.283	3.6×	3.1×	305
50 000	31.179	9.880	3.2×	3.2×	253

Table 2: GPU multiloop (fp32) vs. CPU (fp64) baseline, AMD Radeon 890M, driven spiking neurons, $K = 50\,000$ steps. CPU step is the wall-clock simulation phase (construction removed by a $K = 0$ baseline; low variance, 30+ reps). "GPU step" and "sustained" use the warm steady-state kernel dispatch (reproducible to $\pm 2\%$ for $N \leq 10\,000$); "single-run" is the median of a cold-start run where the CPU-bound model build precedes the first dispatch ($\sim$1.3–2× slower, and up to $\sim$2× more variable). Step-phase speedup peaks near $N = 5000$; beyond $N \approx 10\,000$ the integrated GPU does not sustain its fast clock state under this workload. End-to-end (incl. construction) single-run speedup is 1.2–4.2× (Fig. 2). Raw data: `experiments/data/campaign_warm_dispatch.csv`, `experiments/data/campaign_wallclock_summary.csv`, `experiments/data/campaign_reproducibility.csv`.

under a 2 nA step and compares the membrane voltage across three execution paths (CPU fp64, GPU per-step, GPU multiloop). Over a single action potential the fp32 GPU paths agree with the fp64 CPU to $\sim 2 \times 10^{-7}$ V, and all N neurons remain identical to $< 10^{-6}$ V (Fig. 3; `experiments/data/campaign_divergence.csv`). Over long spiking runs the fp32 and fp64 traces drift in spike *phase*, as any reduced-precision integration of an oscillator must; per-spike waveform and firing rate are preserved. This bounds the accelerator's fidelity honestly and is why the timing comparison above uses driven neurons but the correctness check uses a single-AP window.

CUDA backend on NVIDIA hardware. The CUDA backend builds from the same source with `make USE_CUDA=1` (`nvcc`, `-arch=sm_89`) and is driven by the identical benchmark harness and dispatch/profiling instrumentation. On an NVIDIA RTX 4090 (Ada, driver 570.195, CUDA 12.8, fp32 kernel) the batched multiloop kernel reproduces the fp64 CPU action potential of `hh1952_ap_verify.g` to 7×10^{-8} V with `NEURONS_AGREE: YES`, confirming numerical parity with both the CPU and the OpenCL paths. A validation speedup sweep (Table 3; GPU clocks locked, comparing the GENESIS {`cpu`} step-loop time on an AMD Ryzen 9 9950X against the CUDA multiloop dispatch time)

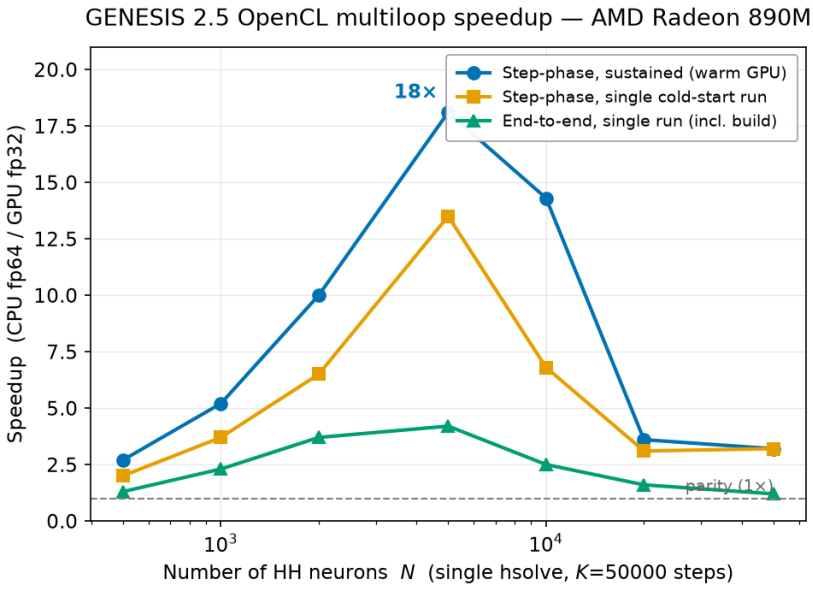

Figure 2: Measured wall-clock speedup, GPU multiloop (fp32) vs. CPU (fp64), after the scheduler and Hines-solver fixes. Step-phase speedup is shown for the GPU's sustained (warm) clock state and for a single cold-start run; the two differ because the CPU-bound model build downclocks the idle GPU before the first dispatch. Both peak near $N = 5000$ and fall beyond $N \approx 10\,000$ as the integrated GPU stops sustaining its fast clocks. End-to-end (single run, including construction) is lower still.

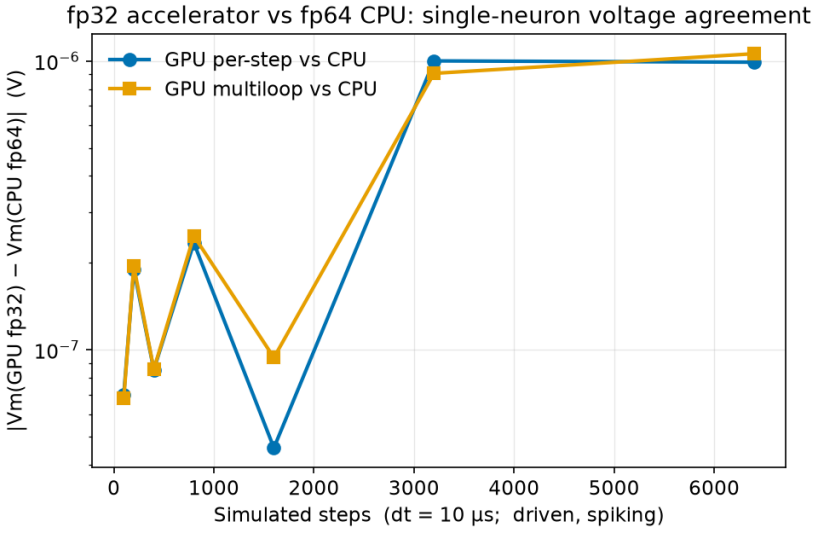

Figure 3: fp32 accelerator vs. fp64 CPU: absolute membrane-voltage difference for a single neuron as a function of simulated steps, for both the per-step and multiloop GPU paths.

shows the step-phase speedup *rising* with network size rather than peaking: unlike the integrated Radeon 890M, the desktop GPU is far from saturated at small N (kernel time is flat at $\sim$48 ms up to $N = 5000$) and keeps scaling, reaching 132$\times$ at $N = 50\,000$. This is a single-replicate dispatch-based measurement – a hardware-validation sweep, not the multi-replicate wall-clock campaign of Table 2; the full build log and pod environment are in `experiments/cuda_validation/`.

N	**CPU step (s)**	**CUDA dispatch (ms)**	**Step-phase speedup**
500	0.248	48.2	5.1$\times$
5 000	2.467	48.2	51$\times$
50 000	25.70	195.1	132$\times$

Table 3: CUDA backend (fp32) vs. CPU (fp64) on an NVIDIA RTX 4090, $K = 50\,000$ steps, GPU clocks locked, single replicate. CPU step is the GENESIS {`cpu`} step-loop time (host: AMD Ryzen 9 9950X); CUDA dispatch is the multiloop kernel wall time for all K steps in one dispatch. The kernel is flat at $\sim$48 ms up to $N = 5000$ (the GPU is under-utilised there), so speedup rises with N – the opposite of the integrated GPU in Table 2. Numerical parity with the fp64 CPU is 7×10^{-8} V over an action potential. Raw data: `experiments/cuda_validation/cuda_sweep_rtx4090.csv`.

Speedup grows with simulation length. Because model construction is a fixed one-time cost, the end-to-end speedup depends on how long the simulation runs. Sweeping K at fixed $N = 5000$ (Fig. 4) shows the end-to-end speedup rising toward the step-phase ceiling as K increases – i.e., the accelerator pays off most for the long runs that dominate real modeling studies (`experiments/data/campaign_ksweep.csv`).

Reproducing the PGENESIS scaling result. The existing MPI path is unaffected by this release; a strong-scaling sweep ($N = 2400$ HH neurons, $P = 1$–24 ranks) on a 12-core/24-thread host gives near-ideal scaling at $P = 2$ ($E_P = 0.95$), a practical efficiency knee near $P \approx 5$, and a recommended operating range of $P = 4$–8 (3.1–4.0$\times$ speedup, 50–78% efficiency) for this problem size.

Full replication steps, scripts, and raw CSV outputs are in `paper/REPLICATION.md`, `genesis/Scripts/benchmark/`, and the self-contained `experiments/` package (including the CUDA validation and pod environment).

4. Impact

New capability for existing deployments. GENESIS/PGENESIS users with channel-kinetics-heavy models gain a path to GPU acceleration – up to $\sim$18$\times$ on the simulation phase (sustained; $\sim$13$\times$ for a

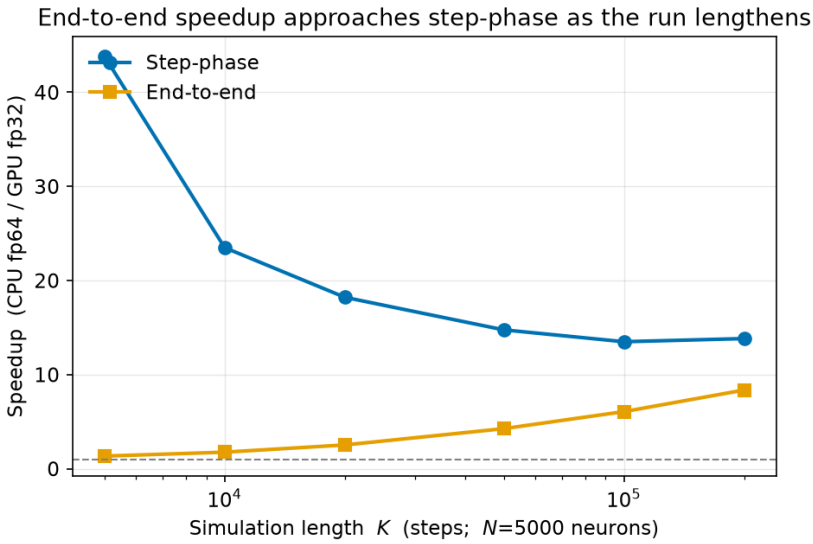

Figure 4: Speedup vs. simulation length K at fixed $N = 5000$. As the run lengthens, the one-time model-construction cost amortizes and the end-to-end speedup approaches the step-phase (simulation-only) speedup.

single cold-start run) and up to ∼4× end-to-end on a laptop-class integrated GPU, batched dispatch mode – without migrating models, SLI scripts, or MPI workflows, the principal barrier that an architecturally different successor (e.g., MOOSE) would otherwise impose.

A correctness fix with effect beyond this release. The three Hines-solver defects fixed here (Section 2) are independent of GPU support and affect any multi-neuron `hsolve` configuration: prior to the fix, `inject`-driven voltage transients were silently suppressed for neurons other than the first in such a configuration, with no error raised. This is a latent correctness issue in the existing GENESIS 2.4 codebase, not introduced by this work, and is reported and fixed for the benefit of any user running multi-neuron `hsolve` models, independent of whether they use the OpenCL backend.

A reusable accelerator-validation discipline. Four confounds masked false readings during this work: a GUI-linked comparison binary, a passive-compartment benchmark that never dispatches to the kernel, CPU-timer semantics that overstate GPU-blocked wait time as compute, and an off-by-one `argc` guard that silently pinned the benchmark step count to its default. The wall-clock protocol established here – both arms timed identically, construction separated from the step phase by a $K = 0$ baseline, device-side dispatch confirmation, and reported

alongside the internal timers they must agree with – generalizes to any future accelerator backend added to GENESIS, and is recorded so that future contributors do not repeat the same false-positive measurements. The CUDA backend, implemented on this same interface and instrumentation, has now been validated on an NVIDIA RTX 4090: a numerical parity check against the fp64 CPU path (agreement 7×10^{-8} V) and a step-phase speedup sweep (5–132×, rising with network size; Table 3). A full multi-replicate wall-clock CUDA campaign and larger-GPU/cluster runs are the natural extension, for which cluster access has been arranged.

Scheduler fix, and the remaining architectural bottleneck. An earlier profiling round found the dominant cost was not GPU dispatch but an $O(N)$ per-step iteration over all absorbed elements in the scheduler (98–99% of GPU-arm step time at $N \geq 2000$). That defect – a broken Hines-numbering loop that left neurons $2 \ldots N$ unabsorbed and re-scheduled every step – has been fixed, and the step-phase speedups reported here are the result. The remaining limits are hardware and structural rather than scheduler overhead: on the integrated GPU tested, the peak speedup is near $N = 5000$ and beyond $N \approx 10\,000$ the device no longer sustains its fast clock state under this workload (the speedup falls toward ∼3×), and the multiloop kernel updates each compartment independently, so it is valid only for single-compartment networks. Extending GPU acceleration to multi-compartment dendritic trees requires a GPU-side tridiagonal (Hines) solve and is recorded as the primary follow-on (`paper/PLAN_gpu_rewrite.md`).

5. Conclusions

   GENESIS 2.5 adds validated, opt-in OpenCL acceleration – and a CUDA port on the same interface – to GENESIS 2.4/PGENESIS 2.4 with no change to existing CPU, MPI, model, or SLI-script workflows. On a laptop-class integrated GPU the batched multi-step kernel delivers up to ∼18× wall-clock speedup on the simulation phase in sustained use (∼13× cold-start; $\sim 1.5 \times 10^9$ neuron-steps/s) and up to ∼4× end-to-end for channel-update-dominated workloads, with the fp32 path matching the fp64 CPU to $\sim 2 \times 10^{-7}$ V over an action potential. The CUDA backend on the same interface is validated on an NVIDIA RTX 4090 (parity to 7×10^{-8} V; 5–132× step-phase speedup, rising with network size). Removing an $O(N)$ scheduler bottleneck is what makes these speedups attainable, and we additionally fix three latent Hines-solver defects affecting any multi-neuron `hsolve` configuration. Recorded next steps: a full multi-replicate wall-clock CUDA campaign on larger and cluster-class GPUs; a GPU-side tridiagonal (Hines) solve

to extend acceleration to multi-compartment dendritic trees; and end-to-end validation on established PGENESIS network models – such as liquid-state-machine reservoirs – to confirm the unmodified-model claim and characterize combined MPI + GPU scaling on cluster hardware.

Acknowledgements

None.

References

[1] Bower JM, Beeman D. *The Book of GENESIS: Exploring Realistic Neural Models with the GEneral NEural SImulation System*. Springer, 1998.

[2] GENESIS 3 Development Plans. `http://www.genesis-sim.org/GENESIS/G3/G3-dev-plans.html`

[3] Cornelis H, Coop AD, Rodriguez AL, Beeman D, Bower JM. GENESIS 3.0 functionality enhanced by interface to multiple types of database. *BMC Neuroscience* 2011, 12(Suppl 1):P15.

[4] Cornelis H, Edwards M, Coop AD, Bower JM. The CBI architecture for computational simulation of realistic neurons and circuits in the GENESIS 3 software federation. *BMC Neuroscience* 2008, 9(Suppl 1):P88.

[5] Cornelis H, Lu H, Esquivel A, Bower JM. Modeling a single dendritic compartment using Neurospaces and GENESIS-3. *BMC Neuroscience* 2007, 8(Suppl 2):P3.

[6] Cornelis H, Bampasakis D, Steuber V, Bower JM. Interoperability in the GENESIS 3.0 Software Federation: the NEURON Simulator as an Example. *BMC Neuroscience* 2013, 14(Suppl 1):P33.

Current executable software version

Not applicable; this release is distributed as source code only (see Current code version above).